

```
CNO nano 7.2
#include <stdio.h>
#include <stdlib.h>

int main() {
    int i;

    // malloc()
    int *a = (int *)malloc(5 * sizeof(int));

    if (a == NULL) {
        printf("malloc failed\n");
        return 1;
    }

    printf("Memory allocated using malloc()\n");

    for (i = 0; i < 5; i++) {
        a[i] = i + 1;
    }

    // calloc()
    int *b = (int *)calloc(5, sizeof(int));

    if (b == NULL) {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("Memory allocated using calloc()\n");

    printf("calloc values: ");
    for (i = 0; i < 5; i++) {
        printf("%d ", b[i]);
    }
    printf("\n");
}
```

```
printf("\n");

// realloc()
a = (int *)realloc(a, 10 * sizeof(int));

if (a == NULL) {
    printf("realloc failed\n");
    free(b);
    return 1;
}

printf("Memory reallocated using realloc()\n");

for (i = 5; i < 10; i++) {
    a[i] = i + 1;
}

printf("Values after realloc: ");
for (i = 0; i < 10; i++) {
    printf("%d ", a[i]);
}
printf("\n");

// free()
free(a);
free(b);

printf("Memory freed using free()\n");

return 0;
}
```

```
sravya@Sravya1955:~/os$ nano memory.c
sravya@Sravya1955:~/os$ gcc memory.c -o memory
sravya@Sravya1955:~/os$ ./memory
Memory allocated using malloc()
Memory allocated using calloc()
calloc values: 0 0 0 0 0
Memory reallocated using realloc()
Values after realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed using free()
sravya@Sravya1955:~/os$ valgrind --version
valgrind-3.22.0
sravya@Sravya1955:~/os$ valgrind --leak-check=full ./memory
==920== Memcheck, a memory error detector
==920== Copyright (C) 2002-2022, and GNU GPL'd, by Julian Seward et al.
==920== Using Valgrind-3.22.0 and LibVEX; rerun with -h for copyright info
==920== Command: ./memory
==920==
Memory allocated using malloc()
Memory allocated using calloc()
calloc values: 0 0 0 0 0
Memory reallocated using realloc()
Values after realloc: 1 2 3 4 5 6 7 8 9 10
Memory freed using free()
==920==
==920== HEAP SUMMARY:
==920==      in use at exit: 0 bytes in 0 blocks
==920==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allocated
==920==
==920== All heap blocks were freed -- no leaks are possible
==920==
==920== For lists of detected and suppressed errors, rerun with: -s
==920== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)
```

GNU nano '7.2

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE (10 * 1024 * 1024)

int main() {
    char *memory;

    // Allocate 10 MB
    memory = (char *)malloc(SIZE);

    if (memory == NULL) {
        perror("malloc");
        return 1;
    }

    // Initialize memory
    for (int i = 0; i < SIZE; i++) {
        memory[i] = 'A';
    }

    printf("Parent: 10 MB memory allocated and initialized.\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        free(memory);
        return 1;
    }

    if (pid == 0) {
        // Child process
        printf("Child: fork completed.\n");
    }
}
```

```

    sleep(2);

    printf("Child: modifying allocated memory...\n");

    // Modify memory
    for (int i = 0; i < SIZE; i++) {
        memory[i] = 'B';
    }

    printf("Child: memory modification completed.\n");

    sleep(5);

    free(memory);
    exit(0);
}

else {
    // Parent process
    printf("Parent: child process created.\n");

    sleep(5);

    printf("Parent: memory still contains '%c'.\n", memory[0]);

    wait(NULL);

    free(memory);

    printf("Parent: child completed and memory freed.\n");
}

return 0;
}

```

```
920 ERROR SUMMARY: 0 errors from 0 contexts
sravya@Sravya1955:~/os$ nano memory.c
sravya@Sravya1955:~/os$ nano cow.c
sravya@Sravya1955:~/os$ gcc cow.c -o cow
sravya@Sravya1955:~/os$ ./cow
Parent: 10 MB memory allocated and initialized
Parent: child process created.
Child: fork completed.
Child: modifying allocated memory...
Child: memory modification completed.
Parent: memory still contains 'A'.
Parent: child completed and memory freed.
sravya@Sravya1955:~/os$ |
```